SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Dry Run Evaluation

Course Code:	CSA17	**Course Title:**	Artificial Intelligence
CO Assessed:	CO1 – Apply (BL3)	**Assessment:**	Assessment Tool 2 – Dry Run Evaluation
Weightage:	35% of CIA	**Duration / Marks:**	60 mins / 100 marks
CO1 Statement:	Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)		
Student Name:	S.Varsha Reddy	**Reg. No.:**	192572177
Section / Batch:	CSE-AI,2025	**Date:**	8/10/2026

Code of Conduct

I S.Varsha Reddy(192572177) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S.Varsha Reddy

Instructions

- Read each question carefully; answer all four questions (25 marks each, 100 in total).
- Show every dry-run step: the initial queue, stack, priority queue or belief state, each iteration, your assumptions and a clear final answer, and finish with the interpretation asked for.
- Follow the conventions stated in each question (alphabetical expansion, explored set, goal test on removal from the frontier, tie-breaking) and present tables neatly.
- Marks follow the Dry Run Evaluation rubric: understanding of the search problem, state expansion and dry run execution, application of the search algorithm, accuracy of the solution path, and presentation and logical reasoning.
- Upload in GitHub and GCR (question paper with Reg. No., Name, answers, rubrics and evaluation page at the end). Duration: 1 hour (15 minutes per question).

Section / Topic	Focus	Question Nos.
1. Foundations and History of AI	Four approaches to AI, Turing Test, eras and milestones	**Q2**
2. Intelligent Agents: Agents and Environments	Agent, percepts, actions, agent function and program, PEAS	**Q1**
3. The Concept of Rationality	Performance measure, percept sequence, rational vs omniscient	**Q1**
4. The Nature of Environments	Task-environment properties and their classification	**Q1**
5. The Structure of Agents	Simple reflex and model-based reflex agent programs	**Q1**
6. Problem-Solving Agents	Goal formulation, problem formulation, search, execution	**Q2**
7. Problem Formulation	States, initial state, actions, goal test, path cost; belief states	**Q2, Q4**
8. Searching for Solutions	Search graph, frontier, explored set, node expansion	**Q3**

9. Uninformed Search Strategies	Dry runs of BFS, DFS and uniform-cost search	**Q3**
10. Search with Partial Information	Sensorless problems, belief states, percept update	**Q4**

Annexure A – Dry Run Evaluation

Question 1: Agents, Environments and Rationality – Dry Run of Agent Programs

Topic: Intelligent agents, rationality, nature of environments, structure of agents | 15 minutes

A vacuum-cleaner agent lives in a world with two squares, A and B. At each time step it perceives [its location, status of its own square] and performs one action: Suck, Right, Left or NoOp. The agent starts in square A, both squares are initially Dirty, and dirt never reappears.

Performance measure over T = 6 time steps: +10 for every square cleaned (a Suck on a Dirty square) and −1 for every Left or Right move.

Agent X (simple reflex): if the current square is Dirty, Suck; otherwise move to the other square. **Agent Y (model-based reflex):** keeps an internal state of the squares it knows to be clean. If the current square is Dirty, Suck; otherwise, if both squares are known clean, NoOp; otherwise move to the other square.

(a) Write the PEAS description (Performance measure, Environment, Actuators, Sensors) of this agent. **[4 Marks]**

(b) Classify the task environment on observability, number of agents, determinism, episodic/sequential, static/dynamic and discrete/continuous, with a one-phrase justification each. **[4 Marks]**

(c) Dry-run both agents for all 6 time steps. Complete the table (percept, action, and Y's internal state after the step). **[9 Marks]**

Step	X: Percept	X: Action	Y: Percept	Y: Action	Y: Known-Clean Squares
1					
2					
3					
4					
5					
6					

(d) Compute the total performance of X and of Y over the 6 steps. **[3 Marks]**

(e) Interpret: which agent is rational? Both agents receive the identical percept [B, Clean] at step 4 – explain why they act differently, using the structure of the two agent programs, and distinguish a rational agent from an omniscient agent. **[5 Marks]**

Question 2: Foundations and History of AI; Problem-Solving Agent and Problem Formulation

Topic: Foundations and history of AI, problem-solving agents, problem formulation | 15 minutes Part A. Complete the table by classifying each system or idea under one of the four approaches to AI (thinking humanly, acting humanly, thinking rationally, acting rationally) and naming the era or milestone of AI history to which it belongs.

(a) Complete the table. **[8 Marks]**

System / Idea	Approach to AI	Era / Milestone
Turing Test proposed (1950)		
General Problem Solver (Newell and Simon, 1957)		
Aristotle's syllogisms and formal logic		
An agent that does the right thing to maximise expected performance		

Part B. A problem-solving agent has two jugs: X of capacity 4 litres and Y of capacity 3 litres, both initially empty and without measuring marks. It may fill a jug from a tap, empty a jug on the ground, or pour one jug into the other until the source is empty or the target is full. The goal is to have exactly 2 litres in jug X.

(b) Formulate the problem: state representation, initial state, actions, transition model, goal test and path cost. **[6 Marks]**

(c) Generate all successor states of (0, 0) and of (4, 0) (leave out actions that leave the state unchanged). **[3 Marks]**

(d) How many states does the state space contain? Write a shortest solution as a sequence of states and give its path cost. **[3 Marks]**

(e) Interpret: describe the four phases (goal formulation, problem formulation, search, execution) of a problem-solving agent for this task, and explain why the agent can execute the found solution without looking at new percepts. **[5 Marks]**

Question 3: Searching for Solutions – Uninformed Search Strategies

Topic: Searching for solutions, BFS, DFS, uniform-cost search | 15 minutes

Consider the weighted undirected graph below. The start node is S and the goal node is G. Use graph search with an explored set; expand neighbours alphabetically, test the goal when a node leaves the frontier, and break ties alphabetically. BFS and DFS ignore edge costs while searching, but path costs are measured with them.

Edge	Cost	Edge	Cost
S – A	1	B – D	1
S – B	4	C – G	6
A – C	2	D – G	2
A – D	5		

(a) Perform BFS from S (queue initially [S]). Do not add a node that is already explored or already in the queue. **[6 Marks]**

Iteration	Node Dequeued	Queue after Expansion	Explored Set
1			
2			
3			
4			
5			
6			

(b) Perform DFS from S using a stack (initially [S]); push children so that the alphabetically first child is popped first. Write the stack with its top on the left. **[5 Marks]**

Iteration	Node Popped	Stack after Pushing Children
1		
2		
3		
4		

(c) Perform uniform-cost search from S. Write queue entries as Node:g(parent) and replace an entry whenever a cheaper path to a frontier node is found. **[7 Marks]**

Iteration	Node Expanded	g(n)	Priority Queue after Expansion
1			
2			
3			
4			
5			
6			

(d) Complete the comparison table. **[4 Marks]**

Strategy	Path Found	Path Cost	Nodes Removed from Frontier	Optimal Here?
BFS				
DFS				

Strategy	Path Found	Path Cost	Nodes Removed from Frontier	Optimal Here?
UCS				

(e) Interpret: explain why UCS removed as many nodes as BFS yet found a cheaper path, why DFS needed the fewest nodes here but cannot be trusted in general, and why an explored set is needed in graph search (use iteration 3 of BFS as an example). **[3 Marks]**

Question 4: Search with Partial Information – Belief States

Topic: Problem formulation, search with partial information (sensorless and percept-based belief-state search) | 15 minutes

A vacuum agent lives in a two-square world (Left square, Right square), each square being Clean (C) or Dirty (D). Actions: Left, Right and Suck (Left in the left square and Right in the right square leave the agent where it is; Suck cleans the current square). Physical states are written (Agent, Left square, Right square):

State	Description	State	Description
S1	(L, D, D)	S5	(L, C, D)
S2	(R, D, D)	S6	(R, C, D)
S3	(L, D, C)	S7	(L, C, C)
S4	(R, D, C)	S8	(R, C, C)

(a) Initially the agent has no sensors and does not know its location or the dirt status. Formulate the problem over belief states: initial belief state, actions, goal test and path cost. **[3 Marks]** **(b)** Apply the action sequence [Right, Suck, Left, Suck] and write the belief state after each action. **[6 Marks]**

Step	Action	Belief State after Action
0	(start)	{S1, S2, S3, S4, S5, S6, S7, S8}
1	Right	
2	Suck	
3	Left	
4	Suck	

(c) Now the agent has a local sensor that reports [location, status of the current square]. Starting again from the initial belief state, the first percept is [Left, Dirty]; the agent then performs Suck and perceives [Left, Clean]; it then performs Right and perceives [Right, Dirty]. Fill in the belief state after each action (prediction) and after each percept (update). **[8 Marks]**

Step	Action	After Action (Predicted)	Percept	After Percept (Updated)
0	(start)	{S1, S2, S3, S4, S5, S6, S7, S8}	[Left, Dirty]	
1	Suck		[Left, Clean]	
2	Right		[Right, Dirty]	

(d) How many different belief states are possible when there are 8 physical states? Why does this make the problem harder than searching over physical states? **[3 Marks]**

(e) Interpret: explain why the percepts in part (c) shrink the belief state, how this differs from the sensorless case in part (b), and why the sensorless plan can still be called rational. **[5 Marks]**

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Accuracy of Solution Path	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

Q. No. / Criteria	Understanding of Search Problem	State Expansion and Dry Run Execution	Application of Search Algorithm	Accuracy of Solution Path	Accuracy of Solution Path	Sub. Total (20)	Total %
1							
2							
3							
4							
Total							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Name:S.Varsha Reddy

Reg No:192572177

1. Ans : Agents, Environments and Rationality

Given Information

- Environment: Two squares, A and B
- Initial location: A
- Initial condition: A = Dirty, B = Dirty
- Dirt never reappears.
- Time steps: T = 6
- Actions: Suck, Right, Left, NoOp
- Performance measure:
    - +10 for cleaning a dirty square
    - −1 for every Left or Right move
    - 0 for NoOp

Agent X – Simple Reflex Agent

- If current square is Dirty → Suck
- Otherwise → move to the other square

Agent Y – Model-Based Reflex Agent

- Maintains an internal record of squares known to be clean.
- If current square is Dirty → Suck
- Otherwise:
    - If both squares are known clean → NoOp
    - Otherwise → move to the other square

(a) PEAS Description

PEAS Component	Description
Performance Measure	+10 for every square cleaned; −1 for every Left/Right movement; NoOp has no cost
Environment	Two-square vacuum world containing squares A and B, initially both dirty
Actuators	Suck, Left, Right, NoOp
Sensors	Sensor perceives the agent's current location and the dirt status of its current square

PEAS Summary

P – Performance: Clean squares efficiently with minimum unnecessary movement.
E – Environment: Two-square vacuum world, A and B.
A – Actuators: Suck, Left, Right, NoOp.
S – Sensors: Location sensor and dirt-status sensor.

(b) Classification of the Task Environment

Property	Classification	Justification
Observability	Partially observable	The agent senses only its current square, not the other square directly.
Number of agents	Single-agent	Only one vacuum-cleaner agent operates in the environment.
Determinism	Deterministic	An action produces a predictable result and dirt never reappears.
Episodic / Sequential	Sequential	The current action affects the agent's location and future decisions.
Static / Dynamic	Static	The environment does not change by itself while the agent is deciding.
Discrete / Continuous	Discrete	There are finite squares, percepts, actions and time steps.

Final Classification

Partially observable, single-agent, deterministic, sequential, static and discrete.

(c) Dry Run of Agents X and Y for 6 Time Steps

Initial State

- Location = A
- A = Dirty
- B = Dirty
- Y's Known-Clean set = ∅

Important assumption for Agent Y

When Y performs Suck on a dirty square, it records that square as clean in its internal state. When it perceives a square as Clean, it also knows that square is clean.

Step-by-Step Dry Run

Step 1

Initial location: A

Percept:

[A, Dirty]

Agent X

- Current square is Dirty.
- Therefore, X performs Suck.
- A becomes Clean.

Agent Y

- Current square is Dirty.
- Therefore, Y performs Suck.
- Y records A as clean.

Y Known-Clean = {A}

Step 2

Location = A and A is Clean.

Percept:

[A, Clean]

Agent X

- A is Clean.
- X does not remember that B is already dirty.
- According to its rule, it moves to the other square.
- Action = Right
- Location becomes B.

Agent Y

- A is Clean.
- Both squares are not known to be clean because only A is known clean.
- Therefore, Y moves to the other square.
- Action = Right
- Location becomes B.

Y Known-Clean = {A}

Step 3

Location = B and B is Dirty.

Percept:

[B, Dirty]

Agent X

- B is Dirty.
- Action = Suck
- B becomes Clean.

Agent Y

- B is Dirty.
- Action = Suck
- Y records B as clean.

Y Known-Clean = {A, B}

Both squares are now clean.

Step 4

Location = B and B is Clean.

Percept:

[B, Clean]

Agent X

- X only considers the current percept.
- It sees that B is Clean.
- According to its simple reflex rule, it moves to the other square.
- Action = Left
- Location becomes A.

Agent Y

- Y sees B is Clean.
- Its internal state says A and B are both clean.
- Therefore, it does not need to move.
- Action = NoOp
- Location remains B.

Y Known-Clean = {A, B}

Step 5

Agent X

Location = A and A is Clean.

Percept:

[A, Clean]

- A is Clean.
- X moves to the other square.
- Action = Right
- Location becomes B.

Agent Y

Location = B and B is Clean.

Percept:

[B, Clean]

- Both A and B are known clean.
- Action = NoOp
- Location remains B.

Y Known-Clean = {A, B}

Step 6

Agent X

Location = B and B is Clean.

Percept:

[B, Clean]

- B is Clean.
- X moves to the other square.
- Action = Left
- Location becomes A.

Agent Y

Location = B and B is Clean.

Percept:

[B, Clean]

- Both squares are known to be clean.
- Action = NoOp
- Location remains B.

Y Known-Clean = {A, B}

Completed Dry-Run Table

Step	X: Percept	X: Action	Y: Percept	Y: Action	Y: Known-Clean Squares
1	[A, Dirty]	Suck	[A, Dirty]	Suck	{A}
2	[A, Clean]	Right	[A, Clean]	Right	{A}
3	[B, Dirty]	Suck	[B, Dirty]	Suck	{A, B}
4	[B, Clean]	Left	[B, Clean]	NoOp	{A, B}
5	[A, Clean]	Right	[B, Clean]	NoOp	{A, B}
6	[B, Clean]	Left	[B, Clean]	NoOp	{A, B}

State Progression

Agent X:

A Dirty → Suck → A Clean → Right → B Dirty → Suck → B Clean → Left → A Clean → Right → B Clean → Left

Agent Y:

A Dirty → Suck → A Clean → Right → B Dirty → Suck → Both Clean Known → NoOp → NoOp → NoOp

(d) Performance Calculation

Agent X

Step	Action	Reward / Cost
1	Suck on Dirty A	+10
2	Right	−1
3	Suck on Dirty B	+10
4	Left	−1
5	Right	−1
6	Left	−1

Total Performance of X

$$10 - 1 + 10 - 1 - 1 - 1$$
$$\boxed{16}$$

Therefore, Performance of X = 16

Agent Y

Step	Action	Reward / Cost
1	Suck on Dirty A	+10
2	Right	−1
3	Suck on Dirty B	+10
4	NoOp	0
5	NoOp	0
6	NoOp	0

Total Performance of Y

$$10 - 1 + 10 + 0 + 0 + 0$$
$$\boxed{19}$$

Therefore,

Performance of Y = 19

Comparison

$$\boxed{Y = 19 > X = 16}$$

Agent Y performs better because it avoids unnecessary movements after both squares have been cleaned.

(e) Interpretation and Rationality

Which Agent is Rational?

Agent Y is more rational in this environment.

Both agents successfully clean the two squares, but Y achieves a higher performance score:

- Agent X = **16**
- Agent Y = **19**

Y receives the same cleaning benefit but avoids unnecessary movement after learning that both squares are clean.

Why do X and Y behave differently at Step 4?

At Step 4, both agents receive the identical percept:

$$\boxed{[B,\ Clean]}$$

However, their internal structures are different.

Agent X – Simple Reflex

Agent X uses only the current percept to choose its action.

It sees:

[B, Clean]

Its rule says:

If the current square is not Dirty, move to the other square.

Therefore:

$$\boxed{X \rightarrow Left}$$

X does not remember that A has already been cleaned.

Agent Y – Model-Based Reflex

Agent Y uses:

1. The current percept, and
2. Its internal model/state.

By Step 3, Y has learned:

$$Known\ Clean = \{A, B\}$$

Therefore, when it receives:

[B, Clean]

it combines the percept with its internal state and determines that both squares are already clean.

Therefore:

$$\boxed{Y \rightarrow NoOp}$$

This demonstrates the main difference between the two agent structures:

Simple reflex agent → acts mainly from current percept

Model-based reflex agent → uses current percept + internal state

Rational Agent vs Omniscient Agent

A rational agent chooses the action that is expected to maximize its performance measure based on the information available to it.

A rational agent does not need to know the future perfectly.

An omniscient agent, on the other hand, would have complete knowledge of the actual state of the environment and the future consequences of its actions.

Therefore:

- Rational: Makes the best decision using available information.
- Omniscient: Knows the actual future outcomes and everything about the environment.

In this problem, Y is more rational because its internal state allows it to avoid unnecessary movement. However, rationality should not be confused with omniscience: Y does not know the future; it simply uses the information it has gathered so far.

Final Answer Summary

Aspect	**Agent X**	**Agent Y**
Agent type	Simple Reflex	Model-Based Reflex
Uses current percept	Yes	Yes
Uses internal state	No	Yes
Squares cleaned	2	2
Unnecessary movements	3	0
Total performance	**16**	**19**
Better agent	—	**Y**

Final Conclusion

Agent Y is the better-performing and more rational agent for this task because it maintains an internal state of known-clean squares and therefore avoids unnecessary movements once both squares are clean. Agent X reacts only to the current percept and consequently continues moving even after the entire environment is clean**.**

2.Ans: Foundations and History of AI; Problem-Solving Agent and Problem Formulation

(a) Classification of Systems / Ideas

The four approaches to AI are:

1. **Thinking Humanly** – machines try to imitate human thought processes.
2. **Acting Humanly** – machines try to behave like humans.
3. **Thinking Rationally** – machines use correct logical reasoning.
4. **Acting Rationally** – machines choose actions that maximize the expected performance measure.

System / Idea	Approach to AI	Era / Milestone
Turing Test proposed (1950)	Acting Humanly	1950 – Turing Test / early AI milestone
General Problem Solver – Newell and Simon (1957)	Thinking Humanly	1950s – Early symbolic AI / cognitive simulation
Aristotle's syllogisms and formal logic	Thinking Rationally	Ancient foundations of AI / formal reasoning
An agent that does the right thing to maximise expected performance	Acting Rationally	Modern AI – Rational-agent approach

Explanation

1. Turing Test – Acting Humanly

The Turing Test evaluates whether a machine can behave in a way that is indistinguishable from a human in conversation.

Therefore:

Acting Humanly

It is an important 1950 milestone in the history of AI.

2. General Problem Solver – Thinking Humanly

The General Problem Solver (GPS), developed by Allen Newell and Herbert A. Simon, attempted to model human problem-solving processes.

Therefore:

Thinking Humanly

It belongs to the early symbolic AI period of the 1950s.

3. Aristotle's Syllogisms – Thinking Rationally

Aristotle developed formal rules of logical reasoning, including syllogisms.

Example:

- All humans are mortal.
- Socrates is a human.
- Therefore, Socrates is mortal.

This represents reasoning according to formal rules.

Therefore:

Thinking Rationally

It represents an ancient foundation of formal logic, which later influenced AI.

4. Maximising Expected Performance – Acting Rationally

A rational agent chooses the action expected to give the best performance based on the information available to it.

Therefore:

Acting Rationally

This represents the modern rational-agent approach to AI.

Part B – Water Jug Problem

Given Information

There are two jugs:

- Jug X capacity = 4 litres
- Jug Y capacity = 3 litres
- Initially both are empty.

- No measuring marks are available.
- Available operations:
    - Fill a jug completely from the tap.
    - Empty a jug onto the ground.
    - Pour one jug into the other until the source is empty or the target is full.
- Goal: exactly 2 litres in Jug X.

(b) Problem Formulation

1. State Representation

Represent a state as:

$$\boxed{(x, y)}$$

where:

- x= amount of water in Jug X
- y= amount of water in Jug Y

Since X has capacity 4 and Y has capacity 3:

$$0 \leq x \leq 4$$
$$0 \leq y \leq 3$$

Example:

$$(4, 2)$$

means:

- Jug X contains 4 litres.
- Jug Y contains 2 litres.

2. Initial State

Both jugs are initially empty.

$$\boxed{(0, 0)}$$

3. Actions

The agent can perform the following actions:

Fill X

$$(x, y) \rightarrow (4, y)$$

Fill Y

$$(x, y) \rightarrow (x, 3)$$

Empty X

$$(x, y) \rightarrow (0, y)$$

Empty Y

$$(x, y) \rightarrow (x, 0)$$

Pour X → Y

Pour water from X into Y until:

- X becomes empty, or
- Y becomes full.

Pour Y → X

Pour water from Y into X until:

- Y becomes empty, or
- X becomes full.

4. Transition Model

The transition model specifies the state obtained after applying an action.

For example:

$$(4, 0) \xrightarrow{Pour\ X \rightarrow Y} (1, 3)$$

because Jug Y can receive only 3 litres:

$$4 - 3 = 1$$

So the resulting state is:

$$\boxed{(1, 3)}$$

5. Goal Test

The goal is to have exactly 2 litres in Jug X.

Therefore:

$$\boxed{x = 2}$$

Any state such as:

$$(2, 0), (2, 1), (2, 2), (2, 3)$$

satisfies the goal.

6. Path Cost

Assumption:

Each valid action has a cost of 1.

Therefore:

$$\boxed{\text{Path cost} = \text{number of actions}}$$

The objective is to find a solution with the minimum number of actions.

(c) Successor States

Successors of Initial State (0,0)

Initial state:

$$\boxed{(0, 0)}$$

We leave out actions that do not change the state.

1. Fill X

$$(0, 0) \rightarrow (4, 0)$$

2. Fill Y

$$(0, 0) \rightarrow (0, 3)$$

3. Empty X

No change:

$$(0,0) \rightarrow (0,0)$$

Therefore, omit it.

4. Empty Y

No change:

$$(0,0) \rightarrow (0,0)$$

Therefore, omit it.

5. Pour X → Y

Both are empty, so no change.

Omit.

6. Pour Y → X

Both are empty, so no change.

Omit.

Therefore:

$$\boxed{Successors(0,0) = \{(4,0),(0,3)\}}$$

Successors of (4,0)

Current state:

$$\boxed{(4,0)}$$

1. Fill X

X is already full.

No change → omit.

2. Fill Y

$$(4,0) \rightarrow (4,3)$$

3. Empty X

$$(4, 0) \rightarrow (0, 0)$$

4. Empty Y

Y is already empty.

No change → omit.

5. Pour X → Y

Y can hold 3 litres.

Therefore:

$$4 - 3 = 1$$

Result:

$$\boxed{(1, 3)}$$

6. Pour Y → X

Y is empty.

No change → omit.

Therefore:

$$\boxed{Successors(4, 0) = \{(4, 3), (0, 0), (1, 3)\}}$$

(d) State Space and Shortest Solution

Number of States

Jug X can contain:

$$0,1,2,3,4$$

So X has:

$$5$$

possible amounts.

Jug Y can contain:

$$0,1,2,3$$

So Y has:

$$4$$

possible amounts.

The theoretical state space contains:

$$5 \times 4 = 20$$

possible pairs.

However, because of the allowed operations, not every pair is reachable from the initial state.

The reachable states are:

$$(0,0), (4,0), (0,3), (4,3),$$
$$(1,3), (0,1), (1,0), (3,3),$$
$$(3,0), (0,2), (4,2), (2,3),$$
$$(2,0), (4,1)$$

Therefore, the reachable state space contains:

$$\boxed{14 \text{ states}}$$

The 20 states are the possible combinations mathematically, while 14 are actually reachable from $(0,0)$using the permitted operations.

Shortest Solution

We use a shortest-path search where every action has cost 1.

One shortest solution is:

Step 1 – Fill X

$$(0,0) \xrightarrow{Fill\ X} (4,0)$$

Step 2 – Pour X → Y

$$(4,0) \xrightarrow{Pour\ X \to Y} (1,3)$$

Three litres move from X to Y.

Step 3 – Empty Y

$$(1,3) \xrightarrow{Empty\ Y} (1,0)$$

Step 4 – Pour X → Y

$$(1,0) \xrightarrow{Pour\ X \to Y} (0,1)$$

The 1 litre in X moves into Y.

Step 5 – Fill X

$$(0,1) \xrightarrow{Fill\ X} (4,1)$$

Step 6 – Pour X → Y

Y already contains 1 litre and can hold only 2 more litres.

Therefore, 2 litres are transferred from X to Y:

$$4 - 2 = 2$$

Result:

$$\boxed{(2,3)}$$

The goal is reached because Jug X contains exactly 2 litres.

Solution Path Table

Step	State	Action	Explanation
0	(0,0)	Initial	Both jugs empty
1	(4,0)	Fill X	X gets 4 L
2	(1,3)	Pour X → Y	3 L transferred
3	(1,0)	Empty Y	Y becomes empty
4	(0,1)	Pour X → Y	1 L transferred
5	(4,1)	Fill X	X gets 4 L
6	(2,3)	Pour X → Y	2 L transferred

Final State

$$\boxed{(2,3)}$$

Therefore:

Jug X contains exactly 2 litres

Path Cost

There are 6 actions:

$$1 + 1 + 1 + 1 + 1 + 1 = \boxed{6}$$

Therefore:

$$\boxed{\text{Shortest path cost} = 6}$$

Dry-Run / Search Reasoning

Since every action has cost 1, Breadth-First Search (BFS) can be used to find a shortest solution.

Initial Frontier

$$Frontier = [(0, 0)]$$

Expand (0,0)

Successors:

$$(4, 0), (0, 3)$$

Frontier:

$$[(4, 0), (0, 3)]$$

Expand (4,0)

New successors:

$$(4, 3), (0, 0), (1, 3)$$

Ignoring the already explored state (0,0):

$$Frontier = [(0, 3), (4, 3), (1, 3)]$$

Continue expansion

BFS continues expanding states level by level until a state with:

$$x = 2$$

is reached.

The shortest goal found is:

$$\boxed{(2, 3)}$$

at depth 6.

Thus:

$$\boxed{\text{Shortest solution cost} = 6}$$

(e) Interpretation – Four Phases of a Problem-Solving Agent

A problem-solving agent generally works through four major phases:

$$\boxed{Goal\ Formulation \rightarrow Problem\ Formulation \rightarrow Search \rightarrow Execution}$$

1. Goal Formulation

First, the agent decides what it wants to achieve.

For this problem:

Have exactly 2 litres of water in Jug X.

Therefore, the goal condition is:

$$\boxed{x = 2}$$

The agent does not initially decide exactly how to reach the goal.

2. Problem Formulation

The agent converts the goal into a formal search problem.

For this jug problem:

Initial state

$$(0, 0)$$

States

$$(x, y)$$

Actions

- Fill X
- Fill Y
- Empty X
- Empty Y
- Pour X → Y
- Pour Y → X

Goal test

$$x = 2$$

Path cost

Each action costs 1.

Thus, the agent has a clearly defined search problem.

3. Search

The agent searches through possible states to find a path from:

$$(0, 0)$$

to any state satisfying:

$$x = 2$$

For example:

$$(0, 0) \rightarrow (4, 0) \rightarrow (1, 3) \rightarrow (1, 0) \rightarrow (0, 1) \rightarrow (4, 1) \rightarrow (2, 3)$$

Since every action has equal cost, BFS can find a shortest solution.

The solution requires:

$$\boxed{6 \text{ actions}}$$

4. Execution

After finding the solution, the agent executes the actions in order:

1. **Fill X**
2. **Pour X → Y**
3. **Empty Y**
4. **Pour X → Y**
5. **Fill X**
6. **Pour X → Y**

Final state:

$$\boxed{(2, 3)}$$

Therefore, the goal is achieved.

Why Can the Agent Execute the Solution Without New Percepts?

This is an important distinction between a problem-solving agent and an agent operating in an uncertain environment.

The jug environment is assumed to be:

- Fully predictable
- Deterministic
- Static
- Known to the agent
- Actions have predictable results.

For example, when the agent is in:

$$(4, 1)$$

and performs:

Pour X → Y

the result is predictably:

$$(2, 3)$$

No unexpected event changes the amount of water.

Therefore, once the agent has found a valid action sequence, it can execute the sequence open-loop, without obtaining new percepts after every action.

The agent can plan the complete solution first because the environment does not unexpectedly change and every action has a known result.

If the jug had leaked, if another person could change the water level, or if the actions had uncertain outcomes, the agent would need new percepts and might have to re-plan.

Final Answer Summary

Part Answer

(a) Turing Test → Acting Humanly; GPS → Thinking Humanly; Aristotle's logic → Thinking Rationally; Expected-performance agent → Acting Rationally

(b) State = (x,y); Initial = (0,0); Goal = x=2; actions = fill, empty, pour; cost = 1/action

(c) Successors of (0,0): (4,0), (0,3); Successors of (4,0): (4,3), (0,0), (1,3)

(d) 20 theoretical combinations; 14 reachable states; shortest path cost = 6

(e) Goal formulation → problem formulation → search → execution; no new percepts are required because the environment is deterministic and predictable

Final Conclusion

The water-jug problem can be represented as a state-space search problem where each state is represented by the amount of water in the two jugs. Starting from (0,0), a shortest solution reaches (2,3) in 6 actions. The problem-solving agent first formulates the goal, represents the problem formally, searches for an optimal path, and finally executes the planned actions. Because the environment is deterministic and the effects of all actions are known, the agent can execute the complete solution without needing new percepts during execution.

3.Ans: SEARCHING FOR SOLUTIONS

Given

Start node:

$$\boxed{S}$$

Goal node:

$$\boxed{G}$$

Edges and costs:

- S–A = 1
- S–B = 4
- A–C = 2
- A–D = 5
- B–D = 1
- C–G = 6
- D–G = 2

Rules Used

1. Neighbours are expanded in alphabetical order.
2. An explored set is maintained.
3. Goal test is performed when a node is removed from the frontier.
4. BFS and DFS ignore edge costs while searching.
5. UCS selects the node with the smallest $g(n)$.
6. Ties are broken alphabetically.

(a) BFS – Breadth-First Search

Initial Queue

$$\boxed{[S]}$$

BFS uses a FIFO queue.

Iteration 1

Remove S.

Neighbours of S:

$$A, B$$

Add them alphabetically.

Queue:

$$[A, B]$$

Explored:

$$\{S\}$$

Iteration 2

Remove **A**.

Neighbours:

$$C, D, S$$

S is already explored, so add C and D.

Queue:

$$[B, C, D]$$

Explored:

$$\{S, A\}$$

Iteration 3

Remove **B**.

Neighbours:

$$D, S$$

- D is already in the queue.
- S is already explored.

Therefore, no new node is added.

Queue:

$$[C, D]$$

Explored:

$$\{S, A, B\}$$

Iteration 4

Remove **C**.

Neighbours:

$$A, G$$

A is already explored. Add G.

Queue:

$$[D, G]$$

Explored:

$$\{S, A, B, C\}$$

Iteration 5

Remove **D**.

Neighbours:

$$A, B, G$$

All are already explored or in the queue.

Queue:

$$[G]$$

Explored:

$$\{S, A, B, C, D\}$$

Iteration 6

Remove **G**.

G is the goal.

$$\boxed{\text{STOP}}$$

BFS Dry-Run Table

Iteration	Node Dequeued	Queue After Expansion	Explored Set
1	S	[A, B]	{S}
2	A	[B, C, D]	{S, A}
3	B	[C, D]	{S, A, B}
4	C	[D, G]	{S, A, B, C}
5	D	[G]	{S, A, B, C, D}
6	G	[]	{S, A, B, C, D, G}

BFS Path

$$\boxed{S \rightarrow A \rightarrow C \rightarrow G}$$

BFS Cost

$$1 + 2 + 6 = \boxed{9}$$

(b) DFS – Depth-First Search

DFS uses a stack.

The question says the stack top is on the left, and the alphabetically first child must be popped first.

Therefore, children are pushed in reverse alphabetical order.

Initial Stack

$$\boxed{[S]}$$

Iteration 1

Pop **S**.

Children:

$$A, B$$

Push B first, then A.

Stack:

$$\boxed{[A, B]}$$

Explored:

$$\{S\}$$

Iteration 2

Pop **A**.

Children:

$$C, D, S$$

S is already explored.

Push D first, then C.

Stack:

$$\boxed{[C, D, B]}$$

Explored:

$$\{S, A\}$$

Iteration 3

Pop **C**.

Children:

$$A, G$$

A is already explored.

Add G.

Stack:

$$\boxed{[G, D, B]}$$

Explored:

$$\{S, A, C\}$$

Iteration 4

Pop **G**.

G is the goal.

$$\boxed{\text{STOP}}$$

DFS Dry-Run Table

Iteration	Node Popped	Stack After Pushing Children
1	S	[A, B]
2	A	[C, D, B]
3	C	[G, D, B]
4	G	[D, B]

DFS Path

$$\boxed{S \rightarrow A \rightarrow C \rightarrow G}$$

DFS Cost

$$1 + 2 + 6 = \boxed{9}$$

(c) Uniform-Cost Search – UCS

UCS always removes the node with the lowest cumulative path cost.

Queue format:

$$\boxed{Node: g(parent)}$$

Iteration 1

Expand **S**.

$$g(S) = 0$$

From S:

$$g(A) = 0 + 1 = 1$$
$$g(B) = 0 + 4 = 4$$

Priority Queue:

$$\boxed{[A, : 1(S) B: 4(S)]}$$

Iteration 2

Expand **A**.

$$g(A) = 1$$

From A:

$$g(C) = 1 + 2 = 3$$
$$g(D) = 1 + 5 = 6$$

Priority Queue:

$$\boxed{[C, :, 3(A) B: 4(S) D: 6(A)]}$$

Iteration 3

Expand **C**.

$$g(C) = 3$$

From C:

$$g(G) = 3 + 6 = 9$$

Priority Queue:

$$\boxed{[B, :, 4(S)D: 6(A)G: 9(C)]}$$

Iteration 4

Expand **B**.

$$g(B) = 4$$

From B:

$$g(D) = 4 + 1 = 5$$

Previously:

$$g(D) = 6$$

Since:

$$5 < 6$$

replace the old entry.

Priority Queue:

$$\boxed{[D, : 5(B)G: 9(C)]}$$

Iteration 5

Expand **D**.

$$g(D) = 5$$

From D:

$$g(G) = 5 + 2 = 7$$

Previously:

$$g(G) = 9$$

Since:

$$7 < 9$$

replace the old entry.

Priority Queue:

$$\boxed{[G: 7(D)]}$$

Iteration 6

Expand **G**.

$$g(G) = 7$$

G is the goal.

$$\boxed{\text{STOP}}$$

UCS Dry-Run Table

Iteration	**Node Expanded**	$g(n)$	**Priority Queue After Expansion**
1	S	0	[A:1(S), B:4(S)]
2	A	1	[C:3(A), B:4(S), D:6(A)]
3	C	3	[B:4(S), D:6(A), G:9(C)]
4	B	4	[D:5(B), G:9(C)]
5	D	5	[G:7(D)]
6	G	7	[]

UCS Path

Using the parent information:

$$G \leftarrow D \leftarrow B \leftarrow S$$

Therefore:

$$\boxed{S \rightarrow B \rightarrow D \rightarrow G}$$

UCS Cost

$$4 + 1 + 2 = \boxed{7}$$

(d) Comparison

Strategy	Path Found	Path Cost	Nodes Removed from Frontier	Optimal Here?
BFS	S → A → C → G	9	6	No
DFS	S → A → C → G	9	4	No
UCS	S → B → D → G	7	6	Yes

Final Result

UCS gives the cheapest path

because:

$$7 < 9$$

(e) Interpretation

1. Why did UCS remove as many nodes as BFS but find a cheaper path?

BFS and UCS both removed 6 nodes, but they use different selection rules.

BFS selects nodes according to their depth and ignores edge costs. Therefore, it found:

$$S \rightarrow A \rightarrow C \rightarrow G$$

with cost:

$$9$$

UCS selects the node having the lowest total path cost $g(n)$. It found:

$$S \rightarrow B \rightarrow D \rightarrow G$$

with cost:

$$7$$

Therefore, UCS found a cheaper path even though it removed the same number of nodes.

2. Why did DFS need the fewest nodes here but cannot be trusted in general?

DFS removed only 4 nodes:

$$S, A, C, G$$

It quickly reached the goal because the first branch happened to contain G.

However, DFS does not consider path cost and is not guaranteed to find the shortest or cheapest solution.

Therefore:

DFS can be fast by chance, but it is not generally optimal.

3. Why is an explored set needed?

The explored set prevents repeated expansion of the same states and avoids cycles.

For example, in BFS Iteration 3, B has neighbours:

$$D, S$$

But:

- D is already in the queue.
- S is already explored.

Therefore, they are not added again.

Without this checking, the search could repeatedly travel through:

$$S \to A \to S \to A \to \cdots$$

Thus:

The explored set prevents duplicate expansion and cycles.

Final Answer

BFS: $S \to A \to C \to G,\ Cost = 9$

DFS: $S \to A \to C \to G,\ Cost = 9$

$$\boxed{\text{UCS: } S \rightarrow B \rightarrow D \rightarrow G,\ Cost = 7}$$

Therefore, UCS is optimal for this graph, while DFS and BFS are not.

4.Ans: Search With Partial Information

Given Information

The vacuum world has two squares:

- Left (L)
- Right (R)

Each square can be:

- C = Clean
- D = Dirty

A physical state is represented as:

$$\boxed{(A, g, ent\ Left\ Right)}$$

Physical States

State	Physical State	Description
S1	(L, D, D)	Agent Left, both Dirty
S2	(R, D, D)	Agent Right, both Dirty
S3	(L, D, C)	Agent Left, Left Dirty, Right Clean
S4	(R, D, C)	Agent Right, Left Dirty, Right Clean
S5	(L, C, D)	Agent Left, Left Clean, Right Dirty
S6	(R, C, D)	Agent Right, Left Clean, Right Dirty
S7	(L, C, C)	Agent Left, both Clean
S8	(R, C, C)	Agent Right, both Clean

Actions

- **Left:** moves the agent to L. If already at L, it remains at L.
- **Right:** moves the agent to R. If already at R, it remains at R.
- **Suck:** cleans the current square. If it is already clean, the state does not change.

(a) Belief-State Problem Formulation

A belief state is the set of physical states that the agent considers possible.

1. Initial Belief State

Initially, the agent has:

- No location information.
- No information about dirt.
- No sensors.

Therefore, all 8 physical states are possible.

$$\boxed{B_0 = \{S, 1, S, 2, S, 3, S, 4S5S6S7S8\}}$$

2. Actions

The available actions are:

$$\boxed{\{L, e, ft\ Right\ Suck\}}$$

The belief-state transition is obtained by applying the action to every physical state in the current belief state and collecting the resulting states.

3. Goal Test

The goal is to have **both squares clean**.

Therefore, every possible physical state in the belief state must have both squares clean.

The only goal states are:

$$S7 = (L, C, C)$$

and

$$S8 = (R, C, C)$$

Thus, the belief-state goal condition is:

$$\boxed{B \subseteq \{S, 7S8\}}$$

4. Path Cost

Assume every action has a cost of 1.

Therefore:

$$\boxed{\text{Path Cost} = \text{Number of Actions}}$$

(b) Sensorless Belief-State Search

The given action sequence is:

$$\boxed{[R, i, g, ht\ Suck\ Left\ Suck]}$$

Since there are **no sensors**, the agent cannot eliminate states based on observations.

Step 0 – Initial Belief State

$$\boxed{B_0 = \{S, 1, S, 2, S, 3, S, 4S5S6S7S8\}}$$

Step 1 – Action: Right

Apply Right to every state.

State transitions

Current State	After Right
S1	S2
S2	S2
S3	S4
S4	S4
S5	S6
S6	S6

S7	S8
S8	S8

Therefore:

$$B_1 = \{S, 2, S, 4S6S8\}$$

Remove duplicates because a belief state is a set.

$$\boxed{B_1 = \{S, 2, S, 4S6S8\}}$$

Step 2 – Action: Suck

Apply Suck to every state in B_1.

State transitions

- S2 = (R,D,D) → S6 = (R,C,D)
- S4 = (R,D,C) → S8 = (R,C,C)
- S6 = (R,C,D) → S6
- S8 = (R,C,C) → S8

Therefore:

$$\boxed{B_2 = \{S, 6S8\}}$$

Step 3 – Action: Left

Apply Left to every state in B_2.

State transitions

- S6 = (R,C,D) → S5 = (L,C,D)
- S8 = (R,C,C) → S7 = (L,C,C)

Therefore:

$$\boxed{B_3 = \{S, 5S7\}}$$

Step 4 – Action: Suck

Apply Suck to both states.

- S5 = (L,C,D) → S5 because Left is already clean.

- S7 = (L,C,C) → S7 because Left is already clean.

Therefore:

$$\boxed{B_4 = \{S, 5S7\}}$$

Completed Table for Part (b)

Step	Action	Belief State After Action
0	Start	{S1, S2, S3, S4, S5, S6, S7, S8}
1	Right	{S2, S4, S6, S8}
2	Suck	{S6, S8}
3	Left	{S5, S7}
4	Suck	{S5, S7}

Important Observation

The final belief state is:

$$\{S, 5S7\}$$

This is not a goal belief state, because S5 still has the Right square dirty:

$$S5 = (L, C, D)$$

Therefore, the given sensorless sequence does not guarantee that both squares are clean.

(c) Belief-State Prediction and Percept Update

Now the agent has a sensor that reports:

$$\boxed{[L, ocation\ Status]}$$

The agent starts again from:

$$B_0 = \{S, 1, S, 2, S, 3, S, 4S5S6S7S8\}$$

The percept sequence is:

1. [Left, Dirty]
2. After Suck → [Left, Clean]
3. After Right → [Right, Dirty]

We must distinguish:

Prediction

Apply the action to every state in the current belief state.

Update

Use the percept to remove states inconsistent with what the sensor observed.

Step 0 – Initial State

Initial Belief

$$\boxed{B_0 = \{S, 1, S, 2, S, 3, S, 4S5S6S7S8\}}$$

First Percept

$$\boxed{[L, eftDirty]}$$

Which states are consistent with this percept?

Only:

$$S1 = (L, D, D)$$

and

$$S3 = (L, D, C)$$

because in both states:

- Agent is at L.
- Left square is Dirty.

Therefore, after the first percept:

$$\boxed{B_0' = \{S, 1S3\}}$$

Step 1 – Action: Suck

Apply Suck to:

$$\{S, 1S3\}$$

State transitions

$$S1 = (L, D, D) \rightarrow S5 = (L, C, D)$$

$$S3 = (L, D, C) \rightarrow S7 = (L, C, C)$$

Therefore, the predicted belief state is:

$$\boxed{B_1^{pred} = \{S, 5S7\}}$$

Percept: [Left, Clean]

Both S5 and S7 are consistent:

- Agent is at L.
- Left square is Clean.

Therefore:

$$\boxed{B_1^{updated} = \{S, 5S7\}}$$

Step 2 – Action: Right

Apply Right to:

$$\{S, 5S7\}$$

State transitions

$$S5 = (L, C, D) \rightarrow S6 = (R, C, D)$$
$$S7 = (L, C, C) \rightarrow S8 = (R, C, C)$$

Therefore, predicted belief state:

$$\boxed{B_2^{pred} = \{S, 6S8\}}$$

Percept: [Right, Dirty]

Now check which states match:

$$S6 = (R, C, D)$$

- Agent = R ✓
- Current square = Right = D ✓

$$S8 = (R, C, C)$$

- Agent = R ✓
- Current square = Right = C ✗

Therefore, S8 is eliminated.

The updated belief state is:

$$\boxed{B_2^{updated} = \{S6\}}$$

Completed Table for Part (c)

Step	Action	After Action (Predicted)	Percept	After Percept (Updated)
0	Start	{S1,S2,S3,S4,S5,S6,S7,S8}	[Left, Dirty]	{S1,S3}
1	Suck	{S5,S7}	[Left, Clean]	{S5,S7}
2	Right	{S6,S8}	[Right, Dirty]	{S6}

(d) Number of Possible Belief States

There are:

$$\boxed{8}$$

physical states.

A belief state is a set of physical states.

For $n$physical states, the number of possible subsets is:

$$2^n$$

Therefore:

$$2^8 = 256$$

Hence:

$$\boxed{256 \text{ possible belief states}}$$

This includes:

- The empty set.
- Single-state belief states.

- Multi-state belief states.
- The complete set of all 8 states.

Why is this harder?

Searching over physical states has only:

$$8$$

states.

But searching over belief states can have:

$$256$$

possible states.

Therefore, the search space becomes much larger.

$$\boxed{8 \text{ physical states} \rightarrow 256 \text{ possible belief states}}$$

This makes search with partial information more difficult because the agent must reason about sets of possible physical states rather than one known physical state.

(e) Interpretation

1. Why do percepts shrink the belief state?

A percept provides information about the actual physical state.

Initially:

$$B = \{S, 1, S, 2, S, 3, S, 4S5S6S7S8\}$$

The first percept is:

$$[L, eftDirty]$$

Only states where:

- Agent is at Left, and
- Left square is Dirty

are possible.

Therefore:

$$\{S, 1, S, 2, S, 3, S, 4S5S6S7S8\}$$

becomes:

$$\boxed{\{S, 1S3\}}$$

The percept eliminates states that are inconsistent with the observation.

Later, after Right, the predicted belief is:

$$\{S, 6S8\}$$

The percept:

$$[R, ightDirty]$$

eliminates S8 because its Right square is Clean.

Therefore:

$$\{S, 6S8\} \rightarrow \boxed{\{S6\}}$$

Thus, percepts reduce uncertainty.

Difference Between Sensorless and Percept-Based Search

Sensorless Case – Part (b)

The agent has no sensors.

Therefore, it cannot determine which physical state is actually true.

For example, after the first action:

$$B = \{S, 2, S, 4S6S8\}$$

The agent must continue planning for all possible states in the belief state.

Percept-Based Case – Part (c)

The agent has a sensor.

Therefore, after each percept, it can eliminate states that are inconsistent with the observation.

For example:

$$\{S, 6S8\}$$

after the Right action becomes:

$$\boxed{\{S6\}}$$

after receiving:

$$[R, ightDirty]$$

Thus:

$$\boxed{\text{Sensorless} \rightarrow \text{uncertainty remains}}$$

$$\boxed{\text{Percept-based} \rightarrow \text{observations reduce uncertainty}}$$

Why Can the Sensorless Plan Still Be Called Rational?

A rational agent does not require complete information.

A rational agent chooses an action that is expected to achieve the goal or maximize its performance based on the information available.

In the sensorless case, the agent has no observations, so it must choose actions that work across the possible states in its belief state.

The sequence:

$$[R, i, g, htSuckLeftSuck]$$

is therefore a rational planned sequence based on the available information, even though the particular sequence given does not guarantee the goal from every possible initial state.

A rational plan must be judged according to the agent's available information and performance objective; lack of sensors does not make rational action impossible.

Final Answer Summary

Part Answer

(a) Initial belief = {S1,S2,S3,S4,S5,S6,S7,S8}; actions = Left, Right, Suck; goal = all possible states have both squares clean; cost = 1/action

Part Answer

(b) After Right = {S2,S4,S6,S8}; after Suck = {S6,S8}; after Left = {S5,S7}; after Suck = {S5,S7}

(c) Initial percept → {S1,S3}; after Suck prediction → {S5,S7}; after [L,C] → {S5,S7}; after Right prediction → {S6,S8}; after [R,D] → {S6}

(d) $2^8 = \boxed{256}$ possible belief states

(e) Percepts reduce uncertainty by eliminating inconsistent physical states; sensorless search must retain all possibilities

Final Conclusion

In a partially observable environment, the agent cannot always know its exact physical state. A belief state represents all states that could currently be true. Without sensors, the agent must propagate the entire set of possibilities through its actions. With sensors, each percept eliminates inconsistent states and therefore reduces uncertainty. Because the number of possible belief states is $2^8 = 256$, belief-state search can be much larger than search over the original 8 physical states.

Name:S.Varsha Reddy

Reg No:192572177

1. Ans : Agents, Environments and Rationality

Given Information

- Environment: Two squares, A and B
- Initial location: A
- Initial condition: A = Dirty, B = Dirty
- Dirt never reappears.
- Time steps: T = 6
- Actions: Suck, Right, Left, NoOp
- Performance measure:
    - +10 for cleaning a dirty square
    - −1 for every Left or Right move
    - 0 for NoOp

Agent X – Simple Reflex Agent

- If current square is Dirty → Suck
- Otherwise → move to the other square

Agent Y – Model-Based Reflex Agent

- Maintains an internal record of squares known to be clean.
- If current square is Dirty → Suck
- Otherwise:
    - If both squares are known clean → NoOp
    - Otherwise → move to the other square

(a) PEAS Description

PEAS Component	Description
Performance Measure	+10 for every square cleaned; −1 for every Left/Right movement; NoOp has no cost
Environment	Two-square vacuum world containing squares A and B, initially both dirty
Actuators	Suck, Left, Right, NoOp
Sensors	Sensor perceives the agent's current location and the dirt status of its current square

PEAS Summary

P – Performance: Clean squares efficiently with minimum unnecessary movement.
E – Environment: Two-square vacuum world, A and B.
A – Actuators: Suck, Left, Right, NoOp.
S – Sensors: Location sensor and dirt-status sensor.

(b) Classification of the Task Environment

Property	Classification	Justification
Observability	Partially observable	The agent senses only its current square, not the other square directly.
Number of agents	Single-agent	Only one vacuum-cleaner agent operates in the environment.
Determinism	Deterministic	An action produces a predictable result and dirt never reappears.
Episodic / Sequential	Sequential	The current action affects the agent's location and future decisions.
Static / Dynamic	Static	The environment does not change by itself while the agent is deciding.
Discrete / Continuous	Discrete	There are finite squares, percepts, actions and time steps.

Final Classification

Partially observable, single-agent, deterministic, sequential, static and discrete.

(c) Dry Run of Agents X and Y for 6 Time Steps

Initial State

- Location = A
- A = Dirty
- B = Dirty
- Y's Known-Clean set = ∅

Important assumption for Agent Y

When Y performs Suck on a dirty square, it records that square as clean in its internal state. When it perceives a square as Clean, it also knows that square is clean.

Step-by-Step Dry Run

Step 1

Initial location: A

Percept:

[A, Dirty]

Agent X

- Current square is Dirty.
- Therefore, X performs Suck.
- A becomes Clean.

Agent Y

- Current square is Dirty.
- Therefore, Y performs Suck.
- Y records A as clean.

Y Known-Clean = {A}

Step 2

Location = A and A is Clean.

Percept:

[A, Clean]

Agent X

- A is Clean.
- X does not remember that B is already dirty.
- According to its rule, it moves to the other square.
- Action = Right
- Location becomes B.

Agent Y

- A is Clean.
- Both squares are not known to be clean because only A is known clean.
- Therefore, Y moves to the other square.
- Action = Right
- Location becomes B.

Y Known-Clean = {A}

Step 3

Location = B and B is Dirty.

Percept:

[B, Dirty]

Agent X

- B is Dirty.
- Action = Suck
- B becomes Clean.

Agent Y

- B is Dirty.
- Action = Suck
- Y records B as clean.

Y Known-Clean = {A, B}

Both squares are now clean.

Step 4

Location = B and B is Clean.

Percept:

[B, Clean]

Agent X

- X only considers the current percept.
- It sees that B is Clean.
- According to its simple reflex rule, it moves to the other square.
- Action = Left
- Location becomes A.

Agent Y

- Y sees B is Clean.
- Its internal state says A and B are both clean.
- Therefore, it does not need to move.
- Action = NoOp
- Location remains B.

Y Known-Clean = {A, B}

Step 5

Agent X

Location = A and A is Clean.

Percept:

[A, Clean]

- A is Clean.
- X moves to the other square.
- Action = Right
- Location becomes B.

Agent Y

Location = B and B is Clean.

Percept:

[B, Clean]

- Both A and B are known clean.
- Action = NoOp
- Location remains B.

Y Known-Clean = {A, B}

Step 6

Agent X

Location = B and B is Clean.

Percept:

[B, Clean]

- B is Clean.
- X moves to the other square.
- Action = Left
- Location becomes A.

Agent Y

Location = B and B is Clean.

Percept:

[B, Clean]

- Both squares are known to be clean.
- Action = NoOp
- Location remains B.

Y Known-Clean = {A, B}

Completed Dry-Run Table

Step	X: Percept	X: Action	Y: Percept	Y: Action	Y: Known-Clean Squares
1	[A, Dirty]	Suck	[A, Dirty]	Suck	{A}
2	[A, Clean]	Right	[A, Clean]	Right	{A}
3	[B, Dirty]	Suck	[B, Dirty]	Suck	{A, B}
4	[B, Clean]	Left	[B, Clean]	NoOp	{A, B}
5	[A, Clean]	Right	[B, Clean]	NoOp	{A, B}
6	[B, Clean]	Left	[B, Clean]	NoOp	{A, B}

State Progression

Agent X:

A Dirty → Suck → A Clean → Right → B Dirty → Suck → B Clean → Left → A Clean → Right → B Clean → Left

Agent Y:

A Dirty → Suck → A Clean → Right → B Dirty → Suck → Both Clean Known → NoOp → NoOp → NoOp

(d) Performance Calculation

Agent X

Step	Action	Reward / Cost
1	Suck on Dirty A	+10
2	Right	−1
3	Suck on Dirty B	+10
4	Left	−1
5	Right	−1
6	Left	−1

Total Performance of X

$$10 - 1 + 10 - 1 - 1 - 1$$
$$\boxed{16}$$

Therefore, Performance of X = 16

Agent Y

Step	Action	Reward / Cost
1	Suck on Dirty A	+10
2	Right	−1
3	Suck on Dirty B	+10
4	NoOp	0
5	NoOp	0
6	NoOp	0

Total Performance of Y

$$10 - 1 + 10 + 0 + 0 + 0$$
$$\boxed{19}$$

Therefore,

Performance of Y = 19

Comparison

$$\boxed{Y = 19 > X = 16}$$

Agent Y performs better because it avoids unnecessary movements after both squares have been cleaned.

(e) Interpretation and Rationality

Which Agent is Rational?

Agent Y is more rational in this environment.

Both agents successfully clean the two squares, but Y achieves a higher performance score:

- Agent X = **16**
- Agent Y = **19**

Y receives the same cleaning benefit but avoids unnecessary movement after learning that both squares are clean.

Why do X and Y behave differently at Step 4?

At Step 4, both agents receive the identical percept:

$$\boxed{[B,\ Clean]}$$

However, their internal structures are different.

Agent X – Simple Reflex

Agent X uses only the current percept to choose its action.

It sees:

[B, Clean]

Its rule says:

If the current square is not Dirty, move to the other square.

Therefore:

$$\boxed{X \rightarrow Left}$$

X does not remember that A has already been cleaned.

Agent Y – Model-Based Reflex

Agent Y uses:

3. The current percept, and
4. Its internal model/state.

By Step 3, Y has learned:

$$Known\ Clean = \{A, B\}$$

Therefore, when it receives:

[B, Clean]

it combines the percept with its internal state and determines that both squares are already clean.

Therefore:

$$\boxed{Y \rightarrow NoOp}$$

This demonstrates the main difference between the two agent structures:

Simple reflex agent → acts mainly from current percept

Model-based reflex agent → uses current percept + internal state

Rational Agent vs Omniscient Agent

A rational agent chooses the action that is expected to maximize its performance measure based on the information available to it.

A rational agent does not need to know the future perfectly.

An omniscient agent, on the other hand, would have complete knowledge of the actual state of the environment and the future consequences of its actions.

Therefore:

- Rational: Makes the best decision using available information.
- Omniscient: Knows the actual future outcomes and everything about the environment.

In this problem, Y is more rational because its internal state allows it to avoid unnecessary movement. However, rationality should not be confused with omniscience: Y does not know the future; it simply uses the information it has gathered so far.

Final Answer Summary

Aspect	**Agent X**	**Agent Y**
Agent type	Simple Reflex	Model-Based Reflex
Uses current percept	Yes	Yes
Uses internal state	No	Yes
Squares cleaned	2	2
Unnecessary movements	3	0
Total performance	**16**	**19**
Better agent	—	**Y**

Final Conclusion

Agent Y is the better-performing and more rational agent for this task because it maintains an internal state of known-clean squares and therefore avoids unnecessary movements once both squares are clean. Agent X reacts only to the current percept and consequently continues moving even after the entire environment is clean**.**

2.Ans: Foundations and History of AI; Problem-Solving Agent and Problem Formulation

(a) Classification of Systems / Ideas

The four approaches to AI are:

5. **Thinking Humanly** – machines try to imitate human thought processes.
6. **Acting Humanly** – machines try to behave like humans.
7. **Thinking Rationally** – machines use correct logical reasoning.
8. **Acting Rationally** – machines choose actions that maximize the expected performance measure.

System / Idea	Approach to AI	Era / Milestone
Turing Test proposed (1950)	Acting Humanly	1950 – Turing Test / early AI milestone
General Problem Solver – Newell and Simon (1957)	Thinking Humanly	1950s – Early symbolic AI / cognitive simulation
Aristotle's syllogisms and formal logic	Thinking Rationally	Ancient foundations of AI / formal reasoning
An agent that does the right thing to maximise expected performance	Acting Rationally	Modern AI – Rational-agent approach

Explanation

1. Turing Test – Acting Humanly

The Turing Test evaluates whether a machine can behave in a way that is indistinguishable from a human in conversation.

Therefore:

Acting Humanly

It is an important 1950 milestone in the history of AI.

2. General Problem Solver – Thinking Humanly

The General Problem Solver (GPS), developed by Allen Newell and Herbert A. Simon, attempted to model human problem-solving processes.

Therefore:

Thinking Humanly

It belongs to the early symbolic AI period of the 1950s.

3. Aristotle's Syllogisms – Thinking Rationally

Aristotle developed formal rules of logical reasoning, including syllogisms.

Example:

- All humans are mortal.
- Socrates is a human.
- Therefore, Socrates is mortal.

This represents reasoning according to formal rules.

Therefore:

Thinking Rationally

It represents an ancient foundation of formal logic, which later influenced AI.

4. Maximising Expected Performance – Acting Rationally

A rational agent chooses the action expected to give the best performance based on the information available to it.

Therefore:

Acting Rationally

This represents the modern rational-agent approach to AI.

Part B – Water Jug Problem

Given Information

There are two jugs:

- Jug X capacity = 4 litres
- Jug Y capacity = 3 litres
- Initially both are empty.

- No measuring marks are available.
- Available operations:
    - Fill a jug completely from the tap.
    - Empty a jug onto the ground.
    - Pour one jug into the other until the source is empty or the target is full.
- Goal: exactly 2 litres in Jug X.

(b) Problem Formulation

1. State Representation

Represent a state as:

$$\boxed{(x, y)}$$

where:

- x= amount of water in Jug X
- y= amount of water in Jug Y

Since X has capacity 4 and Y has capacity 3:

$$0 \leq x \leq 4$$
$$0 \leq y \leq 3$$

Example:

$$(4, 2)$$

means:

- Jug X contains 4 litres.
- Jug Y contains 2 litres.

2. Initial State

Both jugs are initially empty.

$$\boxed{(0, 0)}$$

3. Actions

The agent can perform the following actions:

Fill X

$$(x, y) \rightarrow (4, y)$$

Fill Y

$$(x, y) \rightarrow (x, 3)$$

Empty X

$$(x, y) \rightarrow (0, y)$$

Empty Y

$$(x, y) \rightarrow (x, 0)$$

Pour X → Y

Pour water from X into Y until:

- X becomes empty, or
- Y becomes full.

Pour Y → X

Pour water from Y into X until:

- Y becomes empty, or
- X becomes full.

4. Transition Model

The transition model specifies the state obtained after applying an action.

For example:

$$(4, 0) \xrightarrow{Pour\ X \rightarrow Y} (1, 3)$$

because Jug Y can receive only 3 litres:

$$4 - 3 = 1$$

So the resulting state is:

$$\boxed{(1, 3)}$$

5. Goal Test

The goal is to have exactly 2 litres in Jug X.

Therefore:

$$\boxed{x = 2}$$

Any state such as:

$$(2, 0), (2, 1), (2, 2), (2, 3)$$

satisfies the goal.

6. Path Cost

Assumption:

Each valid action has a cost of 1.

Therefore:

$$\boxed{\text{Path cost} = \text{number of actions}}$$

The objective is to find a solution with the minimum number of actions.

(c) Successor States

Successors of Initial State (0,0)

Initial state:

$$\boxed{(0, 0)}$$

We leave out actions that do not change the state.

1. Fill X

$$(0, 0) \rightarrow (4, 0)$$

2. Fill Y

$$(0, 0) \rightarrow (0, 3)$$

3. Empty X

No change:

$$(0,0) \rightarrow (0,0)$$

Therefore, omit it.

4. Empty Y

No change:

$$(0,0) \rightarrow (0,0)$$

Therefore, omit it.

5. Pour X → Y

Both are empty, so no change.

Omit.

6. Pour Y → X

Both are empty, so no change.

Omit.

Therefore:

$$\boxed{Successors(0,0) = \{(4,0),(0,3)\}}$$

Successors of (4,0)

Current state:

$$\boxed{(4,0)}$$

1. Fill X

X is already full.

No change → omit.

2. Fill Y

$$(4,0) \rightarrow (4,3)$$

3. Empty X

$$(4, 0) \rightarrow (0, 0)$$

4. Empty Y

Y is already empty.

No change → omit.

5. Pour X → Y

Y can hold 3 litres.

Therefore:

$$4 - 3 = 1$$

Result:

$$\boxed{(1, 3)}$$

6. Pour Y → X

Y is empty.

No change → omit.

Therefore:

$$\boxed{Successors(4, 0) = \{(4, 3), (0, 0), (1, 3)\}}$$

(d) State Space and Shortest Solution

Number of States

Jug X can contain:

$$0,1,2,3,4$$

So X has:

$$5$$

possible amounts.

Jug Y can contain:

$$0,1,2,3$$

So Y has:

$$4$$

possible amounts.

The theoretical state space contains:

$$5 \times 4 = 20$$

possible pairs.

However, because of the allowed operations, not every pair is reachable from the initial state.

The reachable states are:

$$(0,0), (4,0), (0,3), (4,3),$$
$$(1,3), (0,1), (1,0), (3,3),$$
$$(3,0), (0,2), (4,2), (2,3),$$
$$(2,0), (4,1)$$

Therefore, the reachable state space contains:

$$\boxed{14 \text{ states}}$$

The 20 states are the possible combinations mathematically, while 14 are actually reachable from $(0,0)$using the permitted operations.

Shortest Solution

We use a shortest-path search where every action has cost 1.

One shortest solution is:

Step 1 – Fill X

$$(0,0) \xrightarrow{Fill\ X} (4,0)$$

Step 2 – Pour X → Y

$$(4,0) \xrightarrow{Pour\ X \to Y} (1,3)$$

Three litres move from X to Y.

Step 3 – Empty Y

$$(1,3) \xrightarrow{Empty\ Y} (1,0)$$

Step 4 – Pour X → Y

$$(1,0) \xrightarrow{Pour\ X \to Y} (0,1)$$

The 1 litre in X moves into Y.

Step 5 – Fill X

$$(0,1) \xrightarrow{Fill\ X} (4,1)$$

Step 6 – Pour X → Y

Y already contains 1 litre and can hold only 2 more litres.

Therefore, 2 litres are transferred from X to Y:

$$4 - 2 = 2$$

Result:

$$\boxed{(2,3)}$$

The goal is reached because Jug X contains exactly 2 litres.

Solution Path Table

Step	State	Action	Explanation
0	(0,0)	Initial	Both jugs empty
1	(4,0)	Fill X	X gets 4 L
2	(1,3)	Pour X → Y	3 L transferred
3	(1,0)	Empty Y	Y becomes empty
4	(0,1)	Pour X → Y	1 L transferred
5	(4,1)	Fill X	X gets 4 L
6	(2,3)	Pour X → Y	2 L transferred

Final State

$$\boxed{(2,3)}$$

Therefore:

Jug X contains exactly 2 litres

Path Cost

There are 6 actions:

$$1 + 1 + 1 + 1 + 1 + 1 = \boxed{6}$$

Therefore:

$$\boxed{\text{Shortest path cost} = 6}$$

Dry-Run / Search Reasoning

Since every action has cost 1, Breadth-First Search (BFS) can be used to find a shortest solution.

Initial Frontier

$$Frontier = [(0, 0)]$$

Expand (0,0)

Successors:

$$(4, 0), (0, 3)$$

Frontier:

$$[(4, 0), (0, 3)]$$

Expand (4,0)

New successors:

$$(4, 3), (0, 0), (1, 3)$$

Ignoring the already explored state (0,0):

$$Frontier = [(0, 3), (4, 3), (1, 3)]$$

Continue expansion

BFS continues expanding states level by level until a state with:

$$x = 2$$

is reached.

The shortest goal found is:

$$\boxed{(2, 3)}$$

at depth 6.

Thus:

$$\boxed{\text{Shortest solution cost} = 6}$$

(e) Interpretation – Four Phases of a Problem-Solving Agent

A problem-solving agent generally works through four major phases:

$$\boxed{Goal\ Formulation \rightarrow Problem\ Formulation \rightarrow Search \rightarrow Execution}$$

1. Goal Formulation

First, the agent decides what it wants to achieve.

For this problem:

Have exactly 2 litres of water in Jug X.

Therefore, the goal condition is:

$$\boxed{x = 2}$$

The agent does not initially decide exactly how to reach the goal.

2. Problem Formulation

The agent converts the goal into a formal search problem.

For this jug problem:

Initial state

$$(0, 0)$$

States

$$(x, y)$$

Actions

- Fill X
- Fill Y
- Empty X
- Empty Y
- Pour X → Y
- Pour Y → X

Goal test

$$x = 2$$

Path cost

Each action costs 1.

Thus, the agent has a clearly defined search problem.

3. Search

The agent searches through possible states to find a path from:

$$(0, 0)$$

to any state satisfying:

$$x = 2$$

For example:

$$(0, 0) \rightarrow (4, 0) \rightarrow (1, 3) \rightarrow (1, 0) \rightarrow (0, 1) \rightarrow (4, 1) \rightarrow (2, 3)$$

Since every action has equal cost, BFS can find a shortest solution.

The solution requires:

$$\boxed{6 \text{ actions}}$$

4. Execution

After finding the solution, the agent executes the actions in order:

7. **Fill X**
8. **Pour X → Y**
9. **Empty Y**
10. **Pour X → Y**
11. **Fill X**
12. **Pour X → Y**

Final state:

$$\boxed{(2, 3)}$$

Therefore, the goal is achieved.

Why Can the Agent Execute the Solution Without New Percepts?

This is an important distinction between a problem-solving agent and an agent operating in an uncertain environment.

The jug environment is assumed to be:

- Fully predictable
- Deterministic
- Static
- Known to the agent
- Actions have predictable results.

For example, when the agent is in:

$$(4, 1)$$

and performs:

Pour X → Y

the result is predictably:

$$(2, 3)$$

No unexpected event changes the amount of water.

Therefore, once the agent has found a valid action sequence, it can execute the sequence open-loop, without obtaining new percepts after every action.

The agent can plan the complete solution first because the environment does not unexpectedly change and every action has a known result.

If the jug had leaked, if another person could change the water level, or if the actions had uncertain outcomes, the agent would need new percepts and might have to re-plan.

Final Answer Summary

Part Answer

(a) Turing Test → Acting Humanly; GPS → Thinking Humanly; Aristotle's logic → Thinking Rationally; Expected-performance agent → Acting Rationally

(b) State = (x,y); Initial = (0,0); Goal = x=2; actions = fill, empty, pour; cost = 1/action

(c) Successors of (0,0): (4,0), (0,3); Successors of (4,0): (4,3), (0,0), (1,3)

(d) 20 theoretical combinations; 14 reachable states; shortest path cost = 6

(e) Goal formulation → problem formulation → search → execution; no new percepts are required because the environment is deterministic and predictable

Final Conclusion

The water-jug problem can be represented as a state-space search problem where each state is represented by the amount of water in the two jugs. Starting from (0,0), a shortest solution reaches (2,3) in 6 actions. The problem-solving agent first formulates the goal, represents the problem formally, searches for an optimal path, and finally executes the planned actions. Because the environment is deterministic and the effects of all actions are known, the agent can execute the complete solution without needing new percepts during execution.

3.Ans: SEARCHING FOR SOLUTIONS

Given

Start node:

$$\boxed{S}$$

Goal node:

$$\boxed{G}$$

Edges and costs:

- S–A = 1
- S–B = 4
- A–C = 2
- A–D = 5
- B–D = 1
- C–G = 6
- D–G = 2

Rules Used

7. Neighbours are expanded in alphabetical order.
8. An explored set is maintained.
9. Goal test is performed when a node is removed from the frontier.
10. BFS and DFS ignore edge costs while searching.
11. UCS selects the node with the smallest $g(n)$.
12. Ties are broken alphabetically.

(a) BFS – Breadth-First Search

Initial Queue

$$\boxed{[S]}$$

BFS uses a FIFO queue.

Iteration 1

Remove S.

Neighbours of S:

$$A, B$$

Add them alphabetically.

Queue:

$$[A, B]$$

Explored:

$$\{S\}$$

Iteration 2

Remove **A**.

Neighbours:

$$C, D, S$$

S is already explored, so add C and D.

Queue:

$$[B, C, D]$$

Explored:

$$\{S, A\}$$

Iteration 3

Remove **B**.

Neighbours:

$$D, S$$

- D is already in the queue.
- S is already explored.

Therefore, no new node is added.

Queue:

$$[C, D]$$

Explored:

$$\{S, A, B\}$$

Iteration 4

Remove **C**.

Neighbours:

$$A, G$$

A is already explored. Add G.

Queue:

$$[D, G]$$

Explored:

$$\{S, A, B, C\}$$

Iteration 5

Remove **D**.

Neighbours:

$$A, B, G$$

All are already explored or in the queue.

Queue:

$$[G]$$

Explored:

$$\{S, A, B, C, D\}$$

Iteration 6

Remove **G**.

G is the goal.

$$\boxed{\text{STOP}}$$

BFS Dry-Run Table

Iteration	Node Dequeued	Queue After Expansion	Explored Set
1	S	[A, B]	{S}
2	A	[B, C, D]	{S, A}
3	B	[C, D]	{S, A, B}
4	C	[D, G]	{S, A, B, C}
5	D	[G]	{S, A, B, C, D}
6	G	[]	{S, A, B, C, D, G}

BFS Path

$$\boxed{S \rightarrow A \rightarrow C \rightarrow G}$$

BFS Cost

$$1 + 2 + 6 = \boxed{9}$$

(b) DFS – Depth-First Search

DFS uses a stack.

The question says the stack top is on the left, and the alphabetically first child must be popped first.

Therefore, children are pushed in reverse alphabetical order.

Initial Stack

$$\boxed{[S]}$$

Iteration 1

Pop **S**.

Children:

$$A, B$$

Push B first, then A.

Stack:

$$\boxed{[A, B]}$$

Explored:

$$\{S\}$$

Iteration 2

Pop **A**.

Children:

$$C, D, S$$

S is already explored.

Push D first, then C.

Stack:

$$\boxed{[C, D, B]}$$

Explored:

$$\{S, A\}$$

Iteration 3

Pop **C**.

Children:

$$A, G$$

A is already explored.

Add G.

Stack:

$$\boxed{[G, D, B]}$$

Explored:

$$\{S, A, C\}$$

Iteration 4

Pop **G**.

G is the goal.

$$\boxed{\text{STOP}}$$

DFS Dry-Run Table

Iteration	Node Popped	Stack After Pushing Children
1	S	[A, B]
2	A	[C, D, B]
3	C	[G, D, B]
4	G	[D, B]

DFS Path

$$\boxed{S \rightarrow A \rightarrow C \rightarrow G}$$

DFS Cost

$$1 + 2 + 6 = \boxed{9}$$

(c) Uniform-Cost Search – UCS

UCS always removes the node with the lowest cumulative path cost.

Queue format:

$$\boxed{Node: g(parent)}$$

Iteration 1

Expand **S**.

$$g(S) = 0$$

From S:

$$g(A) = 0 + 1 = 1$$
$$g(B) = 0 + 4 = 4$$

Priority Queue:

$$\boxed{[A, : 1(S) B: 4(S)]}$$

Iteration 2

Expand **A**.

$$g(A) = 1$$

From A:

$$g(C) = 1 + 2 = 3$$
$$g(D) = 1 + 5 = 6$$

Priority Queue:

$$\boxed{[C, :, 3(A) B: 4(S) D: 6(A)]}$$

Iteration 3

Expand **C**.

$$g(C) = 3$$

From C:

$$g(G) = 3 + 6 = 9$$

Priority Queue:

$$\boxed{[B, :, 4(S)D: 6(A)G: 9(C)]}$$

Iteration 4

Expand **B**.

$$g(B) = 4$$

From B:

$$g(D) = 4 + 1 = 5$$

Previously:

$$g(D) = 6$$

Since:

$$5 < 6$$

replace the old entry.

Priority Queue:

$$\boxed{[D, : 5(B)G: 9(C)]}$$

Iteration 5

Expand **D**.

$$g(D) = 5$$

From D:

$$g(G) = 5 + 2 = 7$$

Previously:

$$g(G) = 9$$

Since:

$$7 < 9$$

replace the old entry.

Priority Queue:

$$\boxed{[G: 7(D)]}$$

Iteration 6

Expand **G**.

$$g(G) = 7$$

G is the goal.

$$\boxed{\text{STOP}}$$

UCS Dry-Run Table

Iteration	**Node Expanded**	$g(n)$	**Priority Queue After Expansion**
1	S	0	[A:1(S), B:4(S)]
2	A	1	[C:3(A), B:4(S), D:6(A)]
3	C	3	[B:4(S), D:6(A), G:9(C)]
4	B	4	[D:5(B), G:9(C)]
5	D	5	[G:7(D)]
6	G	7	[]

UCS Path

Using the parent information:

$$G \leftarrow D \leftarrow B \leftarrow S$$

Therefore:

$$\boxed{S \rightarrow B \rightarrow D \rightarrow G}$$

UCS Cost

$$4 + 1 + 2 = \boxed{7}$$

(d) Comparison

Strategy	Path Found	Path Cost	Nodes Removed from Frontier	Optimal Here?
BFS	S → A → C → G	9	6	No
DFS	S → A → C → G	9	4	No
UCS	S → B → D → G	7	6	Yes

Final Result

UCS gives the cheapest path

because:

$$7 < 9$$

(e) Interpretation

1. Why did UCS remove as many nodes as BFS but find a cheaper path?

BFS and UCS both removed 6 nodes, but they use different selection rules.

BFS selects nodes according to their depth and ignores edge costs. Therefore, it found:

$$S \rightarrow A \rightarrow C \rightarrow G$$

with cost:

$$9$$

UCS selects the node having the lowest total path cost $g(n)$. It found:

$$S \rightarrow B \rightarrow D \rightarrow G$$

with cost:

$$7$$

Therefore, UCS found a cheaper path even though it removed the same number of nodes.

2. Why did DFS need the fewest nodes here but cannot be trusted in general?

DFS removed only 4 nodes:

$$S, A, C, G$$

It quickly reached the goal because the first branch happened to contain G.

However, DFS does not consider path cost and is not guaranteed to find the shortest or cheapest solution.

Therefore:

DFS can be fast by chance, but it is not generally optimal.

3. Why is an explored set needed?

The explored set prevents repeated expansion of the same states and avoids cycles.

For example, in BFS Iteration 3, B has neighbours:

$$D, S$$

But:

- D is already in the queue.
- S is already explored.

Therefore, they are not added again.

Without this checking, the search could repeatedly travel through:

$$S \to A \to S \to A \to \cdots$$

Thus:

The explored set prevents duplicate expansion and cycles.

Final Answer

BFS: $S \to A \to C \to G,\ Cost = 9$

DFS: $S \to A \to C \to G,\ Cost = 9$

$$\boxed{\text{UCS: } S \rightarrow B \rightarrow D \rightarrow G,\ Cost = 7}$$

Therefore, UCS is optimal for this graph, while DFS and BFS are not.

4.Ans: Search With Partial Information

Given Information

The vacuum world has two squares:

- Left (L)
- Right (R)

Each square can be:

- C = Clean
- D = Dirty

A physical state is represented as:

$$\boxed{(A, g, ent\ Left\ Right)}$$

Physical States

State	Physical State	Description
S1	(L, D, D)	Agent Left, both Dirty
S2	(R, D, D)	Agent Right, both Dirty
S3	(L, D, C)	Agent Left, Left Dirty, Right Clean
S4	(R, D, C)	Agent Right, Left Dirty, Right Clean
S5	(L, C, D)	Agent Left, Left Clean, Right Dirty
S6	(R, C, D)	Agent Right, Left Clean, Right Dirty
S7	(L, C, C)	Agent Left, both Clean
S8	(R, C, C)	Agent Right, both Clean

Actions

- **Left:** moves the agent to L. If already at L, it remains at L.
- **Right:** moves the agent to R. If already at R, it remains at R.
- **Suck:** cleans the current square. If it is already clean, the state does not change.

(a) Belief-State Problem Formulation

A belief state is the set of physical states that the agent considers possible.

1. Initial Belief State

Initially, the agent has:

- No location information.
- No information about dirt.
- No sensors.

Therefore, all 8 physical states are possible.

$$\boxed{B_0 = \{S, 1, S, 2, S, 3, S, 4S5S6S7S8\}}$$

2. Actions

The available actions are:

$$\boxed{\{L, e, ft\ Right\ Suck\}}$$

The belief-state transition is obtained by applying the action to every physical state in the current belief state and collecting the resulting states.

3. Goal Test

The goal is to have **both squares clean**.

Therefore, every possible physical state in the belief state must have both squares clean.

The only goal states are:

$$S7 = (L, C, C)$$

and

$$S8 = (R, C, C)$$

Thus, the belief-state goal condition is:

$$\boxed{B \subseteq \{S, 7S8\}}$$

4. Path Cost

Assume every action has a cost of 1.

Therefore:

$$\boxed{\text{Path Cost} = \text{Number of Actions}}$$

(b) Sensorless Belief-State Search

The given action sequence is:

$$\boxed{[R, i, g, ht\ Suck\ Left\ Suck]}$$

Since there are **no sensors**, the agent cannot eliminate states based on observations.

Step 0 – Initial Belief State

$$\boxed{B_0 = \{S, 1, S, 2, S, 3, S, 4S5S6S7S8\}}$$

Step 1 – Action: Right

Apply Right to every state.

State transitions

Current State	After Right
S1	S2
S2	S2
S3	S4
S4	S4
S5	S6
S6	S6

S7	S8
S8	S8

Therefore:

$$B_1 = \{S, 2, S, 4S6S8\}$$

Remove duplicates because a belief state is a set.

$$\boxed{B_1 = \{S, 2, S, 4S6S8\}}$$

Step 2 – Action: Suck

Apply Suck to every state in B_1.

State transitions

- S2 = (R,D,D) → S6 = (R,C,D)
- S4 = (R,D,C) → S8 = (R,C,C)
- S6 = (R,C,D) → S6
- S8 = (R,C,C) → S8

Therefore:

$$\boxed{B_2 = \{S, 6S8\}}$$

Step 3 – Action: Left

Apply Left to every state in B_2.

State transitions

- S6 = (R,C,D) → S5 = (L,C,D)
- S8 = (R,C,C) → S7 = (L,C,C)

Therefore:

$$\boxed{B_3 = \{S, 5S7\}}$$

Step 4 – Action: Suck

Apply Suck to both states.

- S5 = (L,C,D) → S5 because Left is already clean.

- S7 = (L,C,C) → S7 because Left is already clean.

Therefore:

$$\boxed{B_4 = \{S, 5S7\}}$$

Completed Table for Part (b)

Step	Action	Belief State After Action
0	Start	{S1, S2, S3, S4, S5, S6, S7, S8}
1	Right	{S2, S4, S6, S8}
2	Suck	{S6, S8}
3	Left	{S5, S7}
4	Suck	{S5, S7}

Important Observation

The final belief state is:

$$\{S, 5S7\}$$

This is not a goal belief state, because S5 still has the Right square dirty:

$$S5 = (L, C, D)$$

Therefore, the given sensorless sequence does not guarantee that both squares are clean.

(c) Belief-State Prediction and Percept Update

Now the agent has a sensor that reports:

$$\boxed{[L, ocation\ Status]}$$

The agent starts again from:

$$B_0 = \{S, 1, S, 2, S, 3, S, 4S5S6S7S8\}$$

The percept sequence is:

4. [Left, Dirty]
5. After Suck → [Left, Clean]
6. After Right → [Right, Dirty]

We must distinguish:

Prediction

Apply the action to every state in the current belief state.

Update

Use the percept to remove states inconsistent with what the sensor observed.

Step 0 – Initial State

Initial Belief

$$\boxed{B_0 = \{S, 1, S, 2, S, 3, S, 4S5S6S7S8\}}$$

First Percept

$$\boxed{[L, eftDirty]}$$

Which states are consistent with this percept?

Only:

$$S1 = (L, D, D)$$

and

$$S3 = (L, D, C)$$

because in both states:

- Agent is at L.
- Left square is Dirty.

Therefore, after the first percept:

$$\boxed{B_0' = \{S, 1S3\}}$$

Step 1 – Action: Suck

Apply Suck to:

$$\{S, 1S3\}$$

State transitions

$$S1 = (L, D, D) \rightarrow S5 = (L, C, D)$$

$$S3 = (L, D, C) \rightarrow S7 = (L, C, C)$$

Therefore, the predicted belief state is:

$$\boxed{B_1^{pred} = \{S, 5S7\}}$$

Percept: [Left, Clean]

Both S5 and S7 are consistent:

- Agent is at L.
- Left square is Clean.

Therefore:

$$\boxed{B_1^{updated} = \{S, 5S7\}}$$

Step 2 – Action: Right

Apply Right to:

$$\{S, 5S7\}$$

State transitions

$$S5 = (L, C, D) \rightarrow S6 = (R, C, D)$$
$$S7 = (L, C, C) \rightarrow S8 = (R, C, C)$$

Therefore, predicted belief state:

$$\boxed{B_2^{pred} = \{S, 6S8\}}$$

Percept: [Right, Dirty]

Now check which states match:

$$S6 = (R, C, D)$$

- Agent = R ✓
- Current square = Right = D ✓

$$S8 = (R, C, C)$$

- Agent = R ✓
- Current square = Right = C ✗

Therefore, S8 is eliminated.

The updated belief state is:

$$\boxed{B_2^{updated} = \{S6\}}$$

Completed Table for Part (c)

Step	Action	After Action (Predicted)	Percept	After Percept (Updated)
0	Start	{S1,S2,S3,S4,S5,S6,S7,S8}	[Left, Dirty]	{S1,S3}
1	Suck	{S5,S7}	[Left, Clean]	{S5,S7}
2	Right	{S6,S8}	[Right, Dirty]	{S6}

(d) Number of Possible Belief States

There are:

$$\boxed{8}$$

physical states.

A belief state is a set of physical states.

For $n$physical states, the number of possible subsets is:

$$2^n$$

Therefore:

$$2^8 = 256$$

Hence:

$$\boxed{256 \text{ possible belief states}}$$

This includes:

- The empty set.
- Single-state belief states.

- Multi-state belief states.
- The complete set of all 8 states.

Why is this harder?

Searching over physical states has only:

$$8$$

states.

But searching over belief states can have:

$$256$$

possible states.

Therefore, the search space becomes much larger.

$$\boxed{8 \text{ physical states} \rightarrow 256 \text{ possible belief states}}$$

This makes search with partial information more difficult because the agent must reason about sets of possible physical states rather than one known physical state.

(e) Interpretation

1. Why do percepts shrink the belief state?

A percept provides information about the actual physical state.

Initially:

$$B = \{S, 1, S, 2, S, 3, S, 4S5S6S7S8\}$$

The first percept is:

$$[L, eftDirty]$$

Only states where:

- Agent is at Left, and
- Left square is Dirty

are possible.

Therefore:

$$\{S, 1, S, 2, S, 3, S, 4S5S6S7S8\}$$

becomes:

$$\boxed{\{S, 1S3\}}$$

The percept eliminates states that are inconsistent with the observation.

Later, after Right, the predicted belief is:

$$\{S, 6S8\}$$

The percept:

$$[R, ightDirty]$$

eliminates S8 because its Right square is Clean.

Therefore:

$$\{S, 6S8\} \rightarrow \boxed{\{S6\}}$$

Thus, percepts reduce uncertainty.

Difference Between Sensorless and Percept-Based Search

Sensorless Case – Part (b)

The agent has no sensors.

Therefore, it cannot determine which physical state is actually true.

For example, after the first action:

$$B = \{S, 2, S, 4S6S8\}$$

The agent must continue planning for all possible states in the belief state.

Percept-Based Case – Part (c)

The agent has a sensor.

Therefore, after each percept, it can eliminate states that are inconsistent with the observation.

For example:

$$\{S, 6S8\}$$

after the Right action becomes:

$$\boxed{\{S6\}}$$

after receiving:

$$[R, ightDirty]$$

Thus:

$$\boxed{\text{Sensorless} \rightarrow \text{uncertainty remains}}$$

$$\boxed{\text{Percept-based} \rightarrow \text{observations reduce uncertainty}}$$

Why Can the Sensorless Plan Still Be Called Rational?

A rational agent does not require complete information.

A rational agent chooses an action that is expected to achieve the goal or maximize its performance based on the information available.

In the sensorless case, the agent has no observations, so it must choose actions that work across the possible states in its belief state.

The sequence:

$$[R, i, g, htSuckLeftSuck]$$

is therefore a rational planned sequence based on the available information, even though the particular sequence given does not guarantee the goal from every possible initial state.

A rational plan must be judged according to the agent's available information and performance objective; lack of sensors does not make rational action impossible.

Final Answer Summary

Part Answer

(a) Initial belief = {S1,S2,S3,S4,S5,S6,S7,S8}; actions = Left, Right, Suck; goal = all possible states have both squares clean; cost = 1/action

Part Answer

(b) After Right = {S2,S4,S6,S8}; after Suck = {S6,S8}; after Left = {S5,S7}; after Suck = {S5,S7}

(c) Initial percept → {S1,S3}; after Suck prediction → {S5,S7}; after [L,C] → {S5,S7}; after Right prediction → {S6,S8}; after [R,D] → {S6}

(d) $2^8 = \boxed{256}$ possible belief states

(e) Percepts reduce uncertainty by eliminating inconsistent physical states; sensorless search must retain all possibilities

Final Conclusion

In a partially observable environment, the agent cannot always know its exact physical state. A belief state represents all states that could currently be true. Without sensors, the agent must propagate the entire set of possibilities through its actions. With sensors, each percept eliminates inconsistent states and therefore reduces uncertainty. Because the number of possible belief states is $2^8 = 256$, belief-state search can be much larger than search over the original 8 physical states.